

Java Programming: From C

Chapter: 03. Enhancing Expressiveness at the Language Level

Prepared by: Chunghyun Lim (Matriculated in 2019)

Last Updated: January 2026



목차

1. 함수 오버로딩
2. StringBuilder

1. 함수 오버로딩

동일한 동작을 여러 함수로 나누는 문제

- 프로그램에서는 비슷한 기능을 여러 형태로 제공해야 하는 경우가 자주 발생한다.

```
void notify_user_simple(const char* message) {  
    printf("[INFO] %s\n", message);  
}  
  
void notify_user_with_title(const char* title, const char* message) {  
    printf("[%s] %s\n", title, message);  
}  
  
void notify_user_with_priority(const char* title, const char* message, int priority) {  
    printf("[URGENT-%d] %s: %s\n", priority, title, message);  
}
```

- C언어에서는 기능은 유사하지만 이름이 다른 함수를 각각 정의해야 한다.
- 함수 이름이 길어지고 동일한 개념의 동작이 여러 이름으로 분산되는 문제를 만든다.

1. 함수 오버로딩

함수 오버로딩의 개념

- 함수 오버로딩(function overloading)이란 같은 이름의 함수를 여러개 정의하되 매개변수 구성이 다른 경우를 허용하는 기능이다.

```
public static void notifyUser(String message) {  
    System.out.println("[INFO] " + message);  
}  
  
public static void notifyUser(String title, String message) {  
    System.out.println "[" + title + "]" + message);  
}  
  
public static void notifyUser(String title, String message, int priority) {  
    System.out.println("[URGENT-" + priority + "]" + title + ": " + message);  
}
```

- 함수 이름은 동일하지만 매개변수의 개수와 타입이 다르므로 서로 다른 함수로 인식된다.

1. 함수 오버로딩

함수 시그니처와 오버로딩 판단 기준

- 함수 시그니처(signature)란 컴파일러가 함수를 구분하기 위해 사용하는 정보의 집합을 의미한다.
- 함수 시그니처는 다음 요소들로 구성된다.
 - 함수 이름
 - 매개변수의 타입
 - 매개변수의 순서
- 함수의 반환형은 함수 시그니처에 포함되지 않는다.

```
public static int calculateAverageScore(int[] scores);  
public static double calculateAverageScore(int[] scores); // 컴파일 에러
```

- 두 함수는 반환 타입만 다르고 함수 이름과 매개변수 목록이 동일하므로 컴파일러는 이를 서로 다른 함수로 구분할 수 없다.
- 함수 오버로딩은 함수 이름이 같고 매개변수 목록이 서로 다를 때만 허용된다.

1. 함수 오버로딩

함수 오버로딩이 위험해지는 경우

- 함수 오버로딩은 편리하지만 의도하지 않은 함수가 호출될 위험도 존재한다.

```
public static int applyDiscount(double percent, int price) {  
    return price - (price * percent / 100);  
}  
  
public static int applyDiscount(int amount, int price) {  
    return (int) (price - amount);  
}  
  
int price = 100000;  
int discountRate = 15; // 15% 할인 의도  
int finalPrice = applyDisCount(discountRate, price); // applyDiscount(int amount, int price) 호출
```

- 자료형의 자동 승격으로 인해 개발자의 의도와는 다른 오버로드 함수가 호출될 수 있다. 이는 컴파일 오류 없이 발생하므로 더욱 위험하다.

1. 함수 오버로딩

코딩 컨벤션: 함수 오버로딩

- 다음과 같은 경우에만 함수 오버로딩을 제한적으로 사용한다.

- 매개변수의 개수가 다른 경우

```
public static double distance(int x1, int y1, int x2, int y2);  
public static double distance(int x1, int y1, int z1, int x2, int y2, int z2);
```

- 암시적 형 변환이 발생해도 의미가 변하지 않는 경우

```
public static double calculateCircleArea(int radius);  
public static double calculateCircleArea(double radius);
```

- 매개변수의 타입이 서로 명확히 구분되어 암시적 형 변환이 발생할 여지가 없는 경우

```
public static String[] findUser(String email);  
public static String[] findUser(int userId);
```

2. StringBuilder

문자열을 자주 합치면 느려지는 이유

- 문자열을 '+'로 계속 합치면 매번 새로운 임시 문자열이 만들어진다.
- 만들어진 임시 문자열들은 언젠가 GC 대상이 된다.
- 문자열을 많이 수정할수록 만들고 버리는 문자열이 늘어난다.

```
String logMessage = "";  
  
for (int i = 0; i < errors.size(); ++i) {  
    logMessage = logMessage + "[ERROR]" + errors.get(i) + "\n";  
}
```


2. StringBuilder

StringBuilder의 역할

- StringBuilder는 문자열을 효율적으로 만들기 위한 도구이다.
- 내부적으로는 char[] 공간을 미리 확보해 두고 그 공간을 채워 나가며 문자열을 구성한다.
- 최종적으로 필요한 순간에만 String을 만들어낸다.

```
StringBuilder stringBuilder = new StringBuilder(128);  
stringBuilder.append("Why not ");  
stringBuilder.append("change ");  
stringBuilder.append("the world?");
```

```
String result = stringBuilder.toString();  
System.out.println(result);
```

2. StringBuilder

StringBuilder 만들기

```
// 파일 제일 위
import java.lang.StringBuilder;

// 사용할 때
StringBuilder stringBuilder = new StringBuilder(128);
```

- StringBuilder를 만들 때 숫자를 지정하면 내부에 미리 확보해 두는 문자열 공간의 크기를 정할 수 있다.

2. StringBuilder

문자열 추가하기: append()

```
StringBuilder.append("Why not ");  
StringBuilder.append("change ");  
StringBuilder.append("the world?");
```

- append()는 문자열을 뒤에 이어 붙인다.

```
StringBuilder.append("Why not ").append("change ").append("the world?");
```

- 반환값은 자기 자신(this)이다.

2. StringBuilder

현재 길이와 전체 용량 확인: `length()` / `capacity()`

```
int length = stringBuilder.length();  
int capacity = stringBuilder.capacity();
```

- `length()`는 현재까지 들어 있는 문자 수를 반환한다.
- `capacity()`는 내부에 미리 확보해 둔 문자열 공간의 크기를 반환한다.

2. StringBuilder

공간이 부족해지면?

```
StringBuilder stringBuilder = new StringBuilder(8);
stringBuilder.append("Why not ");
int length = stringBuilder.length();    // 8
int capacity = stringBuilder.capacity(); // 8

stringBuilder.append("change ");
length = stringBuilder.length();        // 15
capacity = stringBuilder.capacity();     // 18
```

- 미리 확보해 둔 문자열 공간을 초과하면 StringBuilder는 내부 공간을 늘린 뒤 기존 내용을 복사한다.
- 복사가 반복되면 성능 부담이 커질 수 있으므로 처음 만들 때 충분한 크기를 지정하는 것이 좋다.

2. StringBuilder

최종 문자열 얻기: toString()

- StringBuilder는 문자열을 만드는 과정을 담당한다.
- 최종 결과를 String으로 얻고 싶다면 toString()을 호출해야 한다.

```
String result = stringBuilder.toString();  
System.out.println(result);
```

2. StringBuilder

이 외에 자주 사용하는 메서드 원형

```
char charAt(int index);

StringBuilder insert(int offset, String str);
StringBuilder delete(int start, int end);
StringBuilder deleteCharAt(int index);
StringBuilder replace(int start, int end, String str);
```

- StringBuilder는 문자열 수정을 위한 다양한 메서드를 제공하며 대부분의 메서드는 여러 형태로 오버로딩되어 있다.

[다양한 StringBuilder 메서드들 보기 -> [Java 17 API Specification](#)]

2. StringBuilder

StringBuilder를 쓸 때 / 안 쓸 때

- 문자열 연결 횟수가 많지 않다면 문자열 연결 연산자 +만으로도 충분하다.
- 반복문처럼 문자열 연결이 여러 번 반복되면 StringBuilder가 성능과 메모리 측면에서 유리하다.
- 문자열 연결이 많아질수록 StringBuilder를 우선 고려하며 특히 반복문에서는 StringBuilder 사용이 일반적이다.

Thank you

Wishing you an enjoyable and meaningful learning experience.
May you grow into an engineer who learns with joy and shares with others.

For inquiries, contact: potterLim0808@gmail.com

